Applied Software Development Lab

Lab Handout 7: At the end, we shoot 😀 and clean up 😩

Learning objectives

- **Deepening object-oriented programming**
- **Refactoring existing programs**
- **Incremental software development: small changes, testing, and versioning**

Step 1: Cloning your own repository

Today we are working “only” with Python again. The goal is to further improve the existing code (keyword: *refactoring*) and then to extend its functionality.
Before we start making changes, please take another look at the current state of the game.

As always: Open two terminals – one for editing (left), one for running commands (right).

Left terminal (editing)
Clone the repository and change into the working directory:

```
git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm
cd asp-ulm/python
```

Task 1:

Open the file `breakout.py` in your editor and examine the structure of the program.
Focus on the following points:

- As a group, go through the flow of the program together, in particular the main loop at the end of the file.
- Get an overview of the existing classes and their responsibilities.
- Look specifically for global variables (e.g., `screen`, `width`, `height`, `balls`, `running`) and determine where they are used or modified.
- Distinguish which classes are instantiated (`Ball`) and which are only used as a class object with static attributes and methods (`Paddle`).
- In doing so, recap the role of `__init__` and `self` in classes with instances.

Talk to each other until everyone has understood the basic structure. This preparation is important for the refactoring in the next step.

Right terminal (running commands)
Change into the Python directory here as well:

```
cd asp-ulm/python
```

Task 2

Run the game with the command `python breakout.py`.

Compare the observed behavior with what you found out about the code in Task 1. If there are any discrepancies or open questions, discuss them with us before you continue.

Step 2: Refactoring

The previous global variables are moved into a separate class Game. This class goes into a new file `game.py`. From now on, all other classes are also each moved into a file of their own, so that each file contains exactly one piece of functionality and the code remains easy to follow.

The goal is:
- better structure
- fewer global variables **outside of classes**
- preparation for further object-oriented steps

In the end, the file `breakout.py` should contain little more than the main loop.

Task 1
Create the file `game.py` with the following content:

```
 1 import pygame
 2
 3 class Game:
 4     width, height = None, None
 5     screen = None
 6
 7     def init():
 8         Game.width, Game.height = 800, 600
 9         pygame.init()
10         Game.screen = pygame.display.set_mode((Game.width, Game.height))
11
12     def quit():
13         pygame.quit()
```

Task 2: `ball.py` **(reference for the file at the end of the task)**
Create a new file `ball.py` by copying: `cp breakout.py ball.py`

Edit the new file `ball.py` as follows:
- Delete everything except the class `Ball`.
  Tip for **nano**: Hold down the **Shift key** and use the arrow keys to select text.
  Press **Control+K** to delete the selected text.
- Add the necessary `import` lines at the beginning:
  ```
  1 from game import Game
  2 import pygame
  ```
  Note:
  With `from game import Game`, the class Game is imported directly from the module `game.py`.
  This means you can simply write `Game.width` or `Game.screen` in the code.
  If you used `import game` instead, you would have to include the module name every time, e.g., `game.Game.width`.
  Both variants are correct – but here we choose `from ... import ...` to keep the code easier to read.
- Adjust how global variables are accessed: instead of `width`, `height`, or `screen`, you now use `Game.width`, `Game.height`, `Game.screen`.

© Michael Lehn | CC BY-SA 4.0

Reference: `ball.py`

```
from game import Game
import pygame

class Ball:
    def __init__(self, x, y, vx, vy):
        self.x = x
        self.y = y
        self.vx = vx
        self.vy = vy
        self.c = (255, 255, 0)

    def move(self):
        self.x += self.vx
        self.y += self.vy
        if self.x < 0 or self.x > Game.width:
            self.vx *= -1
        if self.y < 0 or self.y > Game.height:
            self.vy *= -1

    def draw(self, surface):
        pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

Task 2: `paddle.py`
Move the class Paddle into a separate file `paddle.py` – analogous to `ball.py`.
Make sure that here, too, global variables are accessed via the class Game.
The position is now no longer initialized directly in the definition, but by calling `Paddle.init()`.

Reference: `paddle.py`

```
from game import Game
import pygame

class Paddle:
    c = (0, 0, 255)
    w, h = 100, 30
    x, y = None, None

    def init():
        Paddle.x, Paddle.y = (Game.width - Paddle.w) // 2, 570

    def left():
        Paddle.x -= 10
        if Paddle.x < 0:
            Paddle.x = 0

    def right():
        Paddle.x += 10
        if Paddle.x + Paddle.w > Game.width:
            Paddle.x = Game.width - Paddle.w

    def draw(surface):
        pygame.draw.rect(surface, Paddle.c,
                         (Paddle.x, Paddle.y, Paddle.w, Paddle.h))
```

© Michael Lehn | CC BY-SA 4.0

Task 3
Clean up the file `breakout.py` so that it contains only the game flow in the main loop. Import the required classes from the files `game.py`, `ball.py`, and `paddle.py`. Again, adjust the access to global variables. At the beginning, initialize the game with `Game.init()` and then call `Paddle.init()`.

Reference: `breakout.py`

```
 1 from game import Game
 2 from ball import Ball
 3 from paddle import Paddle
 4 import time
 5 import pygame
 6
 7 Game.init()
 8 Paddle.init()
 9
10 running = True
11 balls = [Ball(400, 300, -1, 2), Ball(200, 10, 4, 2)]
12 while running:
13     for event in pygame.event.get():
14         if event.type == pygame.QUIT:
15             running = False
16         elif event.type == pygame.KEYDOWN:
17             if event.key == pygame.K_q:
18                 running = False
19     keys = pygame.key.get_pressed()
20     if keys[pygame.K_a]:
21         balls.append(Ball(400, 300, -1, 2))
22     if keys[pygame.K_LEFT]:
23        Paddle.left()
24     if keys[pygame.K_RIGHT]:
25        Paddle.right()
26
27     pygame.Surface.fill(Game.screen, (0, 0, 0))
28     for i in range(0,len(balls)):
29         balls[i].draw(Game.screen)
30         balls[i].move()
31     Paddle.draw(Game.screen)
32     pygame.display.flip()
33     time.sleep(1/30)
34
35 Game.quit()
```

Task 4: Functional test, git commit, and push
Start the game as usual with `python breakout.py`. Check whether the game works as before. To do so, move the paddle all the way to the left and all the way to the right at least once. This runs through all relevant conditions in `Paddle.left()` and `Paddle.right()`. If, for example, you forgot to replace `width` with `Game.width` during refactoring, you will notice it now at the latest. Of course, this test is not complete – but it covers typical errors that can occur during restructuring.

If the game works as expected, add the new files to the repository and make a commit:

```
git add ball.py paddle.py game.py
git commit -a -m "one file does one thing"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 3: Simplifying interfaces

The method `draw` of the class `Ball` (in `ball.py`) currently looks like this:

```
# ...
class Ball:
    # ...

    def draw(self, surface):
        pygame.draw.circle(surface, self.c, (self.x, self.y), 5)
```

It is called in only one place – in the main loop of `breakout.py`:

```
    balls[i].draw(Game.screen)
```

So drawing always happens on `Game.screen`. That makes sense – we have exactly one **game** and one **drawing surface**.
Passing the drawing surface is therefore redundant and makes the code needlessly general.
Our code will become complex soon enough. So: out with everything that is not needed yet.

Task:

1. Simplify the method `draw` in the class `Ball` as follows:
   ```
   # ...
   class Ball:
     # ...

       def draw(self):
           pygame.draw.circle(Game.screen, self.c, (self.x, self.y), 5)
   ```
   Make sure that you now use Game.screen directly in the method draw.
   Also adjust the call in `breakout.py` accordingly: `balls[i].draw()`
2. **Transfer task:**
   The method `draw` of the class Paddle has the same unnecessary flexibility.
   Adjust it in the same way so that it uses `Game.screen` directly – and can then simply be called in `breakout.py` with `Paddle.draw()`.
3. If everything works as expected, commit your changes:
   `git commit -a -m "simplified draw methods"`
   `git push`

© Michael Lehn | CC BY-SA 4.0

Step 4: Reflecting the ball off the walls

It is high time to show a sketch of our playing field with coordinates:
As you may have already noticed (or are noticing now), the origin is at the **top left**, and the **y-axis points downward** – as is customary in Pygame.

The following sketch shows two balls, each moving toward a wall. Without reflection, they would pass through the wall in the next frame. The reflection prevents this – by reversing the direction of motion along the respective axis.

The sketch shows:
- the current center of the ball at `(self.x, self.y)`
- the new, unreflected position `(new_x, new_y)`
- the corresponding velocity vector `(vx, vy)`
- the impact on the left or right wall and the corresponding reflection



Adapt the `move` method in the class `Ball` (in `ball.py`) as follows:

```
13      def move(self):
14          new_x = self.x + self.vx
15          new_y = self.y + self.vy
16
17          if new_x < 0:
18              new_x = -new_x
19              self.vx = -self.vx
20          if new_x > Game.width:
21              new_x = 2 * Game.width - new_x
22              self.vx = -self.vx
23          if new_y < 0:
24
25              self.vy = -self.vy
26          if new_y > Game.height:
27
28              self.vy = -self.vy
29
30          self.x = new_x
31          self.y = new_y
```

© Michael Lehn | CC BY-SA 4.0

Tasks

1. Start the game with `python breakout.py` and observe how the reflection of the ball at the left and right walls has changed.

   You should see that the ball (more precisely: its center) is reflected realistically. Even at high speed, it always stays **inside the drawing surface** – provided we regard the ball as a **point mass**. The ball itself can therefore stick halfway into the wall.

   How do you arrive at the code for the reflection?

   (A) If `new_x < 0`, the ball has penetrated the left wall by $-$new_x. With a reflection, the new x-coordinate lies symmetrically to 0 at $-$new_x.

   (B) If `new_x > Game.width`, the ball has moved beyond the right edge in the x-direction by new_x $-$ Game.width. The reflected x-coordinate lies symmetrically to Game.width at Game.width $-$ (new_x $-$ Game.width) $= 2 \cdot$ Game.width $-$ new_x

   Note on how to improve the reflection (optional):
   If you want to be really precise, the ball would have to bounce off at
   3. `x = 5` **(left wall)**
   4. `x = Game.width - 5` **(right wall)**

   (assuming that the radius of the ball is 5).

   You should also observe that the ball can still **leave the screen at the top and bottom** – we will fix that next.

2. **Transfer task: Reflection at the floor and ceiling**
   Adjust the code in **line 24** and **line 27** so that the ball is also reflected realistically at the top and bottom – analogous to the side walls.

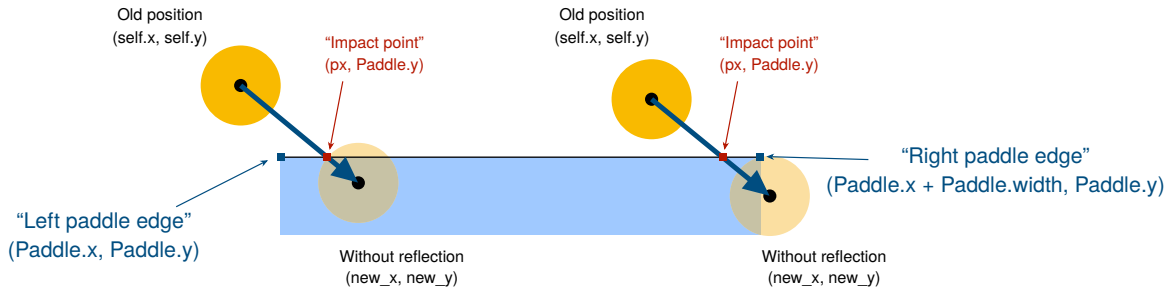
3. If the game works correctly and the ball now also reflects off the ceiling and floor, commit your changes:
   ```
   git commit -a -m "better reflection"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 5: Reflecting the ball off the paddle

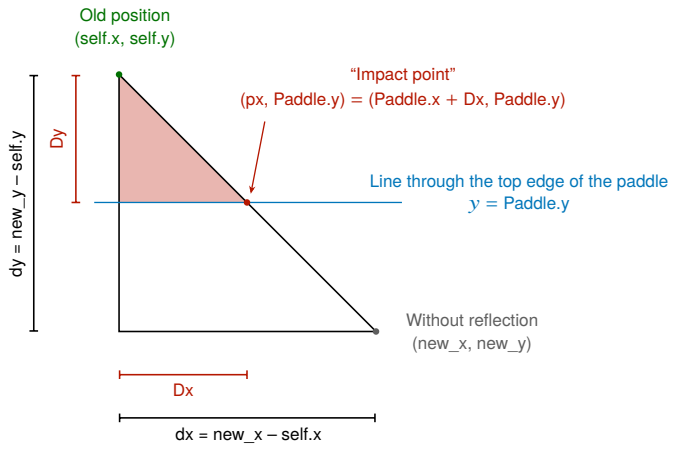
The following sketch shows two situations in which a ball can hit the paddle.
For each ball, the sketch shows its current position `(self.x, self.y)` and the new position `(new_x,new_y)` calculated for the case that the ball **keeps flying unhindered** – that is, **without collision**, as is still the case in the current code:

- The ball on the **left** is initially located to the left of the paddle. Its motion vector crosses the top edge of the paddle, and the new position lies **below the edge and inside the paddle**.
- The ball on the **right** is initially located **above** the paddle. Its new position lies **to the right of the paddle**, but also **below the top edge**.



In both cases: **the motion vector intersects the top edge of the paddle**. The reflection should take place at this **impact point**.

To calculate this intersection point as simply and generally as possible, let us look at the following sketch:



In the sketch you can see two similar triangles – an ideal case for the **good old intercept theorem**. It is simple, intuitive, and leads directly to the solution – and yet it is often forgotten at university or replaced by needlessly complicated calculations. This shows how helpful solid school geometry can be in programming, too.

Our goal: we want to calculate the **x-coordinate of the impact point** – that is, the point at which the motion vector of the ball intersects the top edge of the paddle. To do this, we set up the ratios of corresponding sides of the two triangles:

$$\frac{Dx}{Dy} = \frac{dx}{dy} \quad \text{with} \quad dx = \text{new_x - self.x} \quad \text{and} \quad dy = \text{new_y - self.y}$$

© Michael Lehn | CC BY-SA 4.0

The vertical distance to the paddle edge is:

$$\mathrm{Dy} = \mathrm{Paddle.y} - \mathrm{self.y}$$

Then the horizontal component can be calculated with:

$$\mathrm{Dx} = \frac{\mathrm{dx}}{\mathrm{dy}}\mathrm{Dy}$$

Finally, the x-coordinate of the impact point is:

$$\mathrm{px} = \mathrm{self.x} + \mathrm{Dx}$$

Tasks

1. The following code fragment checks whether the ball crosses the **line through the top edge of the paddle**. If so, the **intersection point** with this edge is to be calculated (you fill in this gap in the code yourself). Then it checks whether the intersection point lies within the **horizontal extent of the paddle**. If so, the ball is reflected by reversing its vertical velocity.

   ```
   if self.y < Paddle.y and new_y >= Paddle.y:
       # Calculate px in this gap
       if px >= Paddle.x and px <= Paddle.x + Paddle.w:
           new_y = Paddle.y + Dy - dy
           self.vy = -self.vy
   ```

   Transfer task: Think about where in the code this fragment should be inserted so that the ball is reflected when it hits the paddle.

   Note for the style-conscious: Use **integer division** (`//`) wherever it makes sense. Even if it does not matter in this small game – deep down, every HPC programmer's heart aches a little at unnecessary floating-point division.

2. If the ball now reflects off the paddle as intended, commit your changes:
   `git commit -a -m "paddle does what it should"`
   `git push`

© Michael Lehn | CC BY-SA 4.0

Step 6: Breakout-style bouncing off the paddle

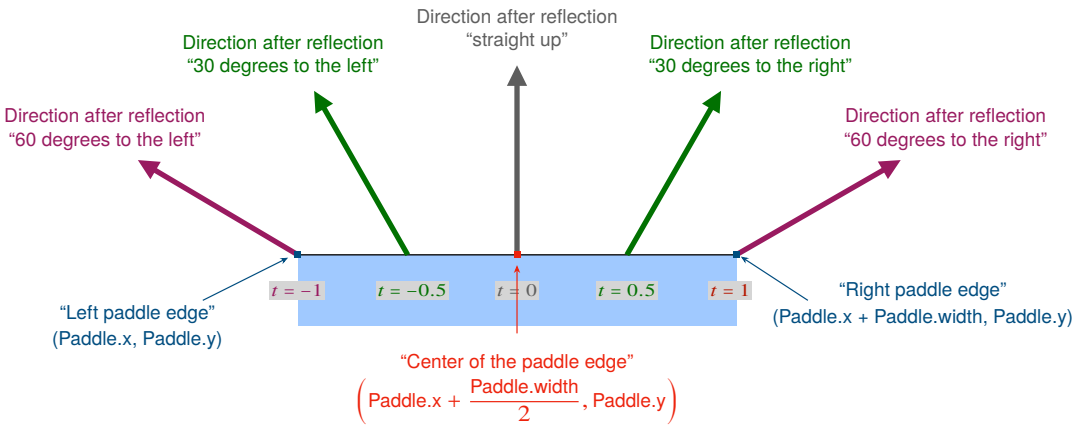
In classic Breakout games, the ball is **not simply reflected** when it hits the paddle. Instead, its new direction of motion depends on **where on the paddle** the ball hits.

A simple model:

- If the ball hits the paddle **exactly in the middle**, it flies **straight up**.
- If it hits the **left edge**, it flies **up and to the left**.
- If it hits the **right edge**, it flies **up and to the right**.

The ball should keep its previous **speed (magnitude)**. Only the **direction** changes – and it does so **linearly depending on the impact point**.

The following sketch illustrates this idea:



We describe the **impact point** on the paddle by a parameter $t \in [-1, 1]$ such that:

- $t = -1$: "far left",
- $t = 0$: "exactly in the middle",
- $t = 1$: "far right".

To do so, we parametrize the x-coordinate of the impact point `px` by

$$\text{px} = \text{Paddle.x} + \frac{\text{Paddle.w}}{2} \cdot (1 + t) \quad \text{with} \quad t \in [-1, 1]$$

Solving for `t` yields:

$$t = (\text{px} - \text{Paddle.x}) \cdot \frac{2}{\text{Paddle.w}} - 1$$

Now we can map the parameter $t \in [-1, 1]$ to an angle φ in **radians**. A few annoying details have to be taken into account – for example, that in our coordinate system the **y-axis points downward**. The following formula already takes this into account correctly:

$$\varphi = \left(\frac{t}{3} - 1\right) \cdot \frac{\pi}{2} \in \left[-\frac{4\pi}{6}, -\frac{2\pi}{6}\right].$$

This means:

- $t = -1$ gives an angle of **60° up to the left.**
- $t = 1$ gives **60° up to the right.**
- $t = 0$ gives **straight up.**

© Michael Lehn | CC BY-SA 4.0

The magnitude of the velocity is preserved:

$$v = \sqrt{\text{self.vx}^2 + \text{self.vy}^2}$$

The new velocity vector is then given by:

$$\text{self.vx} = v \cdot \cos\varphi \quad \text{and} \quad \text{self.vy} = v \cdot \sin\varphi.$$

Tasks

1. **Now let's put the theory into practice again!**

   For the implementation, we need some mathematical functions provided by the module `math`. Import the module in `ball.py`:

   ```
   import math
   ```

   What we need from it:

   - `math.pi`: the constant π
   - `math.hypot(x, y)`: computes $\sqrt{x^2 + y^2}$
   - `math.cos` and `math.sin` compute the cosine and sine of an angle in **radians**.

   With these tools, it should not be difficult to complete the following code fragment and to place it **at a suitable point** in the code:

   ```
   if px >= Paddle.x and px <= Paddle.x + Paddle.w:
       new_y = Paddle.y + Dy - dy
       # Calculate t, phi, v, self.vx, self.vy in this gap
   ```

   Remark: Strictly speaking, we first still reflect the ball according to the principle "angle of incidence = angle of reflection" – and only then change the direction according to the **Breakout rules**. This is not entirely clean, but to the human eye it will look exactly as if we reflected the ball Breakout-style right away.

2. **If everything works, then as always: into the repository!**

   ```
   git commit -a -m "implemented Breakout reflection"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 7: Something without theory for a change

Now we simply make the code a little nicer – without any theory at all.
In `breakout.py`, replace this loop:

```
    for i in range(0,len(balls)):
        balls[i].draw()
        balls[i].move()
```

with the more compact and more readable variant:

```
    for b in balls:
        b.draw()
        b.move()
```

Both variants do exactly the same thing – but the second one is clearer, shorter, and more in line with Python style.

As always: Test whether everything still works as expected. Then off to the repository with
`git commit -a -m "nicer loop"`
`git push`

Step 8: Quickly adding a new feature (rockets!)

We have now spent a lot of time cleaning up our code, structuring it more nicely, and solving details such as the ball reflection properly. High time to check whether all this pays off in practice!

Our paddle now gets a new ability: **firing rockets!**
The rockets should fly upward, i.e., similar to balls, only in the opposite direction.
At first, they are even displayed exactly like balls – we may change that later.

Create a new file `rocket.py`. The easiest way is probably to copy the file `ball.py` and adapt it: `cp ball.py rocket.py` Then replace its content with:

```
from game import Game
from paddle import Paddle
import pygame
import math

class Rocket:
    def __init__(self, x, y, vy):
        self.x = x
        self.y = y
        self.vy = vy
        self.c = (255, 255, 255)

    def move(self):
        if self.y < 10:
            return
        self.y += self.vy

    def draw(self):
        pygame.draw.circle(Game.screen, self.c, (self.x, self.y), 5)
```

© Michael Lehn | CC BY-SA 4.0

Tasks

The following changes in `breakout.py` are sufficient:

1. **Importing the rocket class**

   At the very top, with the imports (preferably next to the other `from ... import ...` lines), add:

   ```
   from rocket import Rocket
   ```

   This lets you write `Rocket(...)` directly later on, instead of `rocket.Rocket(...)`.

2. **Creating the list of rockets**
   Near the place where the balls are initialized (e.g., directly after it), add:

   ```
   rockets = []
   ```

   At first, everything is peaceful – no rockets on the way yet.

3. **Catching the arrow key ↑ and creating a rocket**
   Extend the handling of key-press events (where you already check for `pygame.KEYDOWN` and `event.key == pygame.K_q`) as follows:

   ```
   elif event.type == pygame.KEYDOWN:
       if event.key == pygame.K_q:
           running = False
       elif event.key == pygame.K_UP:
           rockets.append(Rocket(Paddle.x + Paddle.w//2, Paddle.y, -10))
   ```

   The start position is the middle of the paddle, and the velocity is -10 (because our y-axis points downward).

4. **Moving and drawing the rockets**
   So far, the main loop moves and draws all balls – now you do the same for the rockets. Directly afterwards:

   ```
   for r in rockets:
       r.draw()
       r.move()
   ```

   From now on, you can test the feature with `python breakout.py`.

5. **Once all this runs and the rockets fly – into the repository:**

   ```
   git add breakout.py rocket.py
   git commit -m "added rockets"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 9: Finally, we clean up!

Our rockets stick to the ceiling – and on purpose!
So far, the method `move` only checked whether `self.y < 10` and then simply **did nothing more**. This is why they stay right there.
The value `10` was **deliberately** chosen instead of `0` – this way, you can see the "garbage" that our code has produced so far.

Task
Now we clean up: the class `Rocket` gets a new attribute `alive`. When a rocket reaches the ceiling (i.e., `y < 10`), this attribute is set to `False` – and we then remove the rocket from the list `rockets`.

New class Rocket in `rocket.py`:

```
class Rocket:
    def __init__(self, x, y, vy):
        self.x = x
        self.y = y
        self.vy = vy
        self.c = (255, 255, 255)
        self.alive = True

    def move(self):
        if self.y < 10:
            self.alive = False
            return
        self.y += self.vy

    def draw(self):
        pygame.draw.circle(Game.screen, self.c, (self.x, self.y), 5)
```

In the main loop of `breakout.py`
add the following line after drawing and moving the rockets:

```
    rockets = [item for item in rockets if item.alive]
```

What does this line do?
This is a so-called **list comprehension**. It creates a new list that **contains only the rockets** for which `alive == True` – that is, those that have not yet "exploded" at the ceiling. **In short:** "Keep only the living rockets."

If everything works, you can improve the `move` method of a rocket a little: set `alive` to False as soon as the y-coordinate is less than `0`.

Attention: Afterwards, be sure to test once more whether everything still works correctly – to rule out typical last-minute typos.

Then into the repository:

```
git add rocket.py breakout.py
git commit -m "rockets disappear at the ceiling"
git push
```

Final remark:
In the same way, you can also make balls disappear that you miss with the paddle. That is where we will start next time – to pick up right where we left off today.

© Michael Lehn | CC BY-SA 4.0